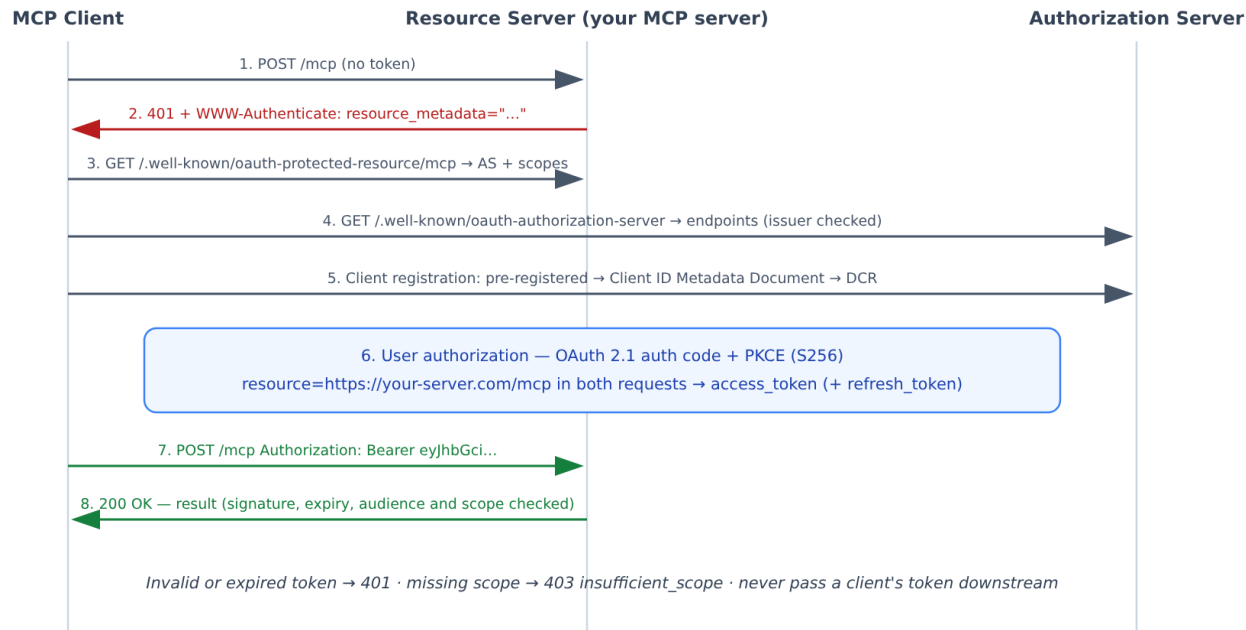


Section 1 – The OAuth 2.1 Resource-Server Model

In MCP, your server is a Resource Server - it holds the protected tools and resources. A separate Authorization Server (AS) issues tokens. The client discovers how to get a token, obtains one that is issued specifically for your server, and then presents it as a Bearer token on every HTTP request.

OAuth 2.1 Resource-Server Model in MCP

The client discovers where to get a token, then presents it as a Bearer token on every request



The discovery-to-token flow. The client learns where to authenticate, obtains a token, and retries the original request.

Step 1 – Submit request without Token.

The client has no idea yet that the server is protected, so it simply sends its first MCP request with no Authorization header. This “fail first, then discover” pattern is deliberate: the server, not the client, is the authority on what authentication it requires.

HTTP – Unauthenticated request

```
POST /mcp HTTP/1.1
Host: your-server.com
Content-Type: application/json
MCP-Protocol-Version: 2026-07-28
Mcp-Method: server/discover
```

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "server/discover",
  "params": {
    "_meta": {
      "io.modelcontextprotocol.protocolVersion": "2026-07-28",
      ...
    }
  }
}
```

Step 2 - The server challenges with 401 + where to find auth info

HTTP - 401 challenge response

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer error="invalid_token",
  resource_metadata="https://your-server.com/.well-known/oauth-protected-resource/mcp",
  scope="mcp:tools"
```

The scope parameter tells the client which scopes to ask for. If the header carries no resource_metadata, the client falls back to the well-known URIs: first /.well-known/oauth-protected-resource/mcp (the path of the MCP endpoint), then /.well-known/oauth-protected-resource at the root.

Step 3 - Protected Resource Metadata (PRM) discovery

The client fetches the pointer to learn the Authorization Server and the scopes it supports.

JSON - Protected Resource Metadata

```
{
  "resource": "https://your-server.com/mcp",
  "authorization_servers": ["https://auth.your-server.com"],
  "scopes_supported": ["mcp:tools", "mcp:resources"],
  "bearer_methods_supported": ["header"]
}
```

Step 4 - Authorization Server discovery

The client fetches the AS metadata, trying /.well-known/oauth-authorization-server first and /.well-known/openid-configuration second. It only uses the document if its issuer matches the AS URL it started from.

JSON - Authorization Server metadata

```
{
  "issuer": "https://auth.your-server.com",
  "authorization_endpoint": "https://auth.your-server.com/authorize",
  "token_endpoint": "https://auth.your-server.com/token",
  "code_challenge_methods_supported": ["S256"],
  "client_id_metadata_document_supported": true,
```

```
"registration_endpoint": "https://auth.your-server.com/register"
}
```

Step 5 - Client registration

The client needs a `client_id`. It tries, in order: (1) a client ID that was pre-registered with this AS; (2) a Client ID Metadata Document - the `client_id` is an HTTPS URL (for example `https://app.example.com/oauth/client.json`) that points to a JSON document with `client_id`, `client_name` and `redirect_uris`, used when the AS advertises `client_id_metadata_document_supported`; (3) Dynamic Client Registration at the `registration_endpoint`.

Step 6 - User authorization

Standard OAuth 2.1 authorization code flow with PKCE (S256). The client sends `resource=https://your-server.com/mcp` (RFC 8707) in both the authorization request and the token request, so the token is bound to this one server. When the browser returns, the client checks that the `iss` value matches the AS it chose, then exchanges the code for tokens (`access_token`, `refresh_token`, `token_type` Bearer, `expires_in`).

Step 7 - Authenticated requests

The access token rides in the Authorization header of every HTTP request (never in the query string):

HTTP - authenticated request

```
POST /mcp HTTP/1.1
Host: your-server.com
Authorization: Bearer eyJhbGciOiJSUzI1NiIs...
```

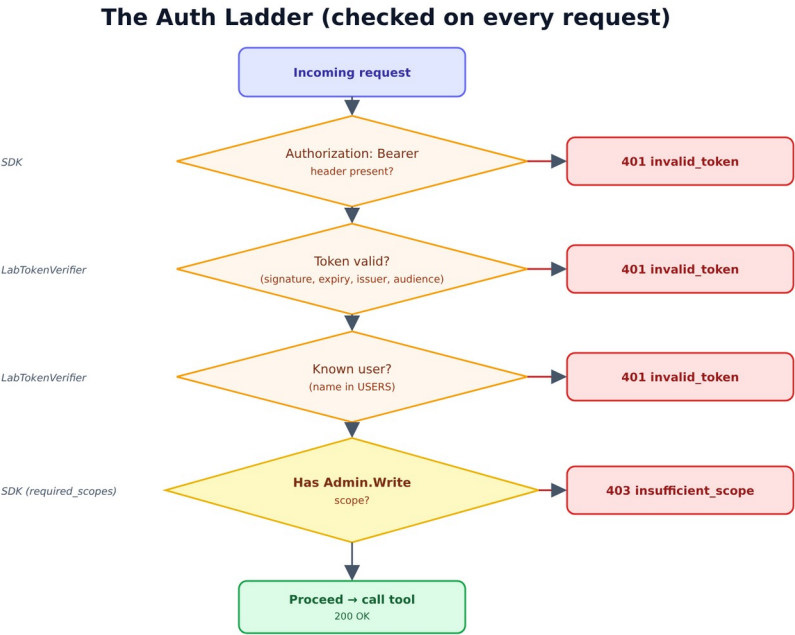
The server must check that the token is valid, unexpired and issued for it (its audience). Invalid or expired tokens get 401. A valid token that lacks a needed scope gets 403 with WWW-Authenticate: Bearer error="insufficient_scope", scope="..." - the client then re-authorizes, asking for its current scopes plus the new ones (step-up authorization), and retries. The server never forwards the client's token to other services.

Section 2 - The Lab's Simplification (read this first)

This lab keeps the resource-server enforcement - the part you own - and simplifies token issuance to an HS256 JWT minted locally by util.py. The MCP SDK does the HTTP plumbing (the 401/403 challenges and the Protected Resource Metadata endpoint); you supply the token check.

Production	Lab
External Auth Server	util.py mints the token
RS256	HS256
JWKS key rotation	Shared secret
Full OAuth flow (PKCE, resource, refresh)	Signature, expiry, issuer and audience check

On every request the server climbs a ladder: Bearer header present → token valid (signature, expiry, issuer, audience) → known user → has the Admin.Write scope. The first three failures return 401 invalid_token; a missing scope returns 403 insufficient_scope.



The enforcement ladder. The first failed rung decides the status code; later rungs never run.

Hands-On Lab — A Secured MCP Server

The SecurityDemo folder contains five Python files. HowJWTWorks.py is a standalone learning aid; the other four are the running system.

File	What it does
HowJWTWorks.py	A standalone script – no MCP involved. Mints a JWT, validates it, and shows the audience check rejecting a token meant for another server. Run this first.
util.py	The stand-in Authorization Server. generate_token() mints an HS256 JWT carrying iss, aud, name, exp and scope, and python util.py writes it to .env. Flags create failing tokens for testing: --no-admin-scope, --expired, --user, --audience. validate_token() checks signature, expiry, issuer and audience.
token_verifier.py	The ladder rungs you own. LabTokenVerifier is called by the SDK for every request; it checks the token and the user and returns an AccessToken (or None, which becomes 401).
server.py	The MCP Resource Server. Exposes one tool, get_time, and turns on the SDK's bearer-token protection with token_verifier= and AuthSettings (required scope, resource URL). The SDK also publishes the Protected Resource Metadata.
client.py	A Streamable HTTP client that reads TOKEN from .env, attaches it as a Bearer header on every request, and prints any 401/403 challenge.

Example:

1. git clone <https://github.com/DeccansoftAITeam/MCPDemos.git>
2. cd SecurityDemo
3. python -m venv venv
4. venv\Scripts\activate (Windows)
5. source venv/bin/activate (macOS / Linux)
6. pip install -r requirements.txt
7. python HowJWTWorks.py
8. Terminal1:
 - a. python util.py
 - b. python server.py
9. Terminal 2:
 - a. python client.py

Code walkthrough

Four ideas carry the whole lab. Read them in this order.

1. util.py – what a token actually asserts

A JWT is only a signed dictionary. Everything the server later decides is already written into this payload:

```
now = datetime.now(timezone.utc)
payload = {
    "iss": ISSUER,          # issuer
    "sub": "1234567890",    # who
    "name": name,           # identity rung
    "aud": audience,        # audience rung (RFC 8707 resource)
    "client_id": "lab-client", # which client app asked for it
    "iat": now,
    "exp": now + timedelta(minutes=minutes), # expiry rung
    "scope": " ".join(scopes), # authorisation rung (space-separated, OAuth style)
}
return jwt.encode(payload, SECRET_KEY, algorithm="HS256")
```

The aud (audience) claim names the one server allowed to accept this token, and validate_token() passes audience= and issuer= to jwt.decode, so a token minted for another server is refused. Because the lab signs with a symmetric HS256 secret, the same SECRET_KEY must be known to both the minter and the validator – which is why token_verifier.py imports validate_token from util.py. In production the AS signs with a private key (RS256/ES256) and the server verifies with the public key from the AS's JWKS endpoint.

2. token_verifier.py – the rungs you own

The SDK checks the Bearer header itself, then calls verify_token(). Returning None makes the SDK answer 401 invalid_token. Returning an AccessToken hands the scopes to the SDK, which then checks Admin.Write and answers 403 insufficient_scope if it is missing.

```
USERS = {"User Userson", "Admin Adminson"}

class LabTokenVerifier(TokenVerifier):
    async def verify_token(self, token: str) → AccessToken | None:
        claims = validate_token(token)    # rung 2: signature + expiry + issuer + audience
        if claims is None:
            print("→ Invalid or expired token!")
            return None
```

```

if claims.get("name") not in USERS: # rung 3: known identity
    print("→ User does not exist!")
    return None

print(f"→ Token OK for {claims['name']} (scopes: {claims.get('scope', '')})")
return AccessToken(          # the scopes the SDK checks in rung 4
    token=token,
    client_id=claims.get("client_id", "unknown"),
    scopes=claims.get("scope", "").split(),
    expires_at=claims["exp"],
    resource=claims["aud"],
    subject=claims["sub"],
    claims=claims,
)

```

Until every rung passes, the MCP request body is never parsed and no tool is ever reached.

3. server.py – turning protection on

Authentication here is transport-level, not per-tool. You pass the verifier and AuthSettings to MCPServer; the SDK then wraps the Streamable HTTP endpoint with bearer-token middleware and publishes /.well-known/oauth-protected-resource/mcp:

```

mcp = MCPServer(
    name="MCP Resource Server",
    instructions="Returns the server time to callers holding the Admin.Write scope.",
    token_verifier=LabTokenVerifier(),
    auth=AuthSettings(
        issuer_url=ISSUER,          # where clients get tokens (advertised in PRM)
        resource_server_url=RESOURCE_URL, # this server's identity = expected token audience
        required_scopes=["Admin.Write"], # rung 4
        validate_token_resource=True,    # SDK re-checks the audience (rung 2)
    ),
)

# ... tools ...

```

```
mcp.run(transport="streamable-http", host="localhost", port=8000)
```

This is the single most important structural point in the module: one setting protects every tool, resource and prompt the server will ever expose, including ones added later, so the check cannot be forgotten on a new tool. Inside a tool, `get_access_token()` returns the validated token - `get_time` uses it to report who called.

4. client.py – the token on every request

The header is set on the HTTP client, not on a single call, because the specification requires the Authorization header on every HTTP request from client to server – even for requests that are part of the same logical session:

```
token = os.getenv("TOKEN")
http_client = httpx2.AsyncClient(
    headers={"Authorization": f"Bearer {token}"},
    event_hooks={"response": [show_auth_errors]}, # prints 401/403 challenges
)

async with http_client, Client(streamable_http_client(URL, http_client=http_client)) as client:
    result = await client.call_tool("get_time", {})
    print("get_time →", result.structured_content)
```

Try deleting the headers argument: the very first request (server/discover) fails with 401, before `call_tool` is ever reached.

Verify the auth ladder (tested behaviour)

Request	Server logs	Result
No Authorization header	(no log line – rejected before the verifier runs)	401 invalid_token
Valid admin token (python util.py)	→ Token OK for User Userson (scopes: Admin.Write User.Read)	get_time returns the time and caller
python util.py --no-admin-scope	→ Token OK for User Userson (scopes: User.Read)	403 insufficient_scope
python util.py --expired	[FAIL] Token has expired. → Invalid or expired token!	401 invalid_token
Header present but not "Bearer <token>"	(no log line – rejected before the verifier runs)	401 invalid_token
python util.py --user "Eve"	→ User does not exist!	401 invalid_token

python util.py --audience http://other-server/mcp	[FAIL] Invalid token: Audience doesn't match	401 invalid_token
--	---	-------------------

Done when

- util.py writes TOKEN to .env.
- With the default (admin) token, client.py prints get_time with the time and caller User Userson.
- No header, expired, unknown-user or wrong-audience token → 401 invalid_token; token lacking Admin.Write → 403 insufficient_scope (check the server log lines).
- curl http://localhost:8000/.well-known/oauth-protected-resource/mcp returns the Protected Resource Metadata (resource, authorization_servers, scopes_supported).
- You can place each check on the auth ladder and say which HTTP status it returns.